

TP INTÉGRATION CONTINUE

Mise en place d'un pipeline CI/CD

Jenkins & SonarQube

Audit et sécurisation d'un microservice d'authentification Java

Khalid Filali-Mdarhri · Cyprien Broche · Yanis Benadrouche

Dépôt : github.com/Kfilalimdarhri/Projet-CI

S	M	Nom du projet 1	Dernier succès	Dernier échec	Dernière durée	
...	☀	pipeline-ci	s. o.	s. o.	ND	▶

Le pipeline « pipeline-ci » créé dans Jenkins, point d'entrée de toute la chaîne CI/CD.

1. Contexte & objectif

L'équipe de développement a livré, dans l'urgence, un microservice d'authentification en Java (Maven). Le code fonctionne mais contient des failles de sécurité et de mauvaises pratiques. Notre mission, en tant qu'ingénieurs DevOps, comporte quatre volets :

- Construire un pipeline d'intégration continue avec Jenkins ;
- Interconnecter SonarQube pour auditer le code automatiquement à chaque push ;
- Corriger les failles identifiées pour passer le Quality Gate ;
- (Bonus) Conteneuriser l'application avec Docker.

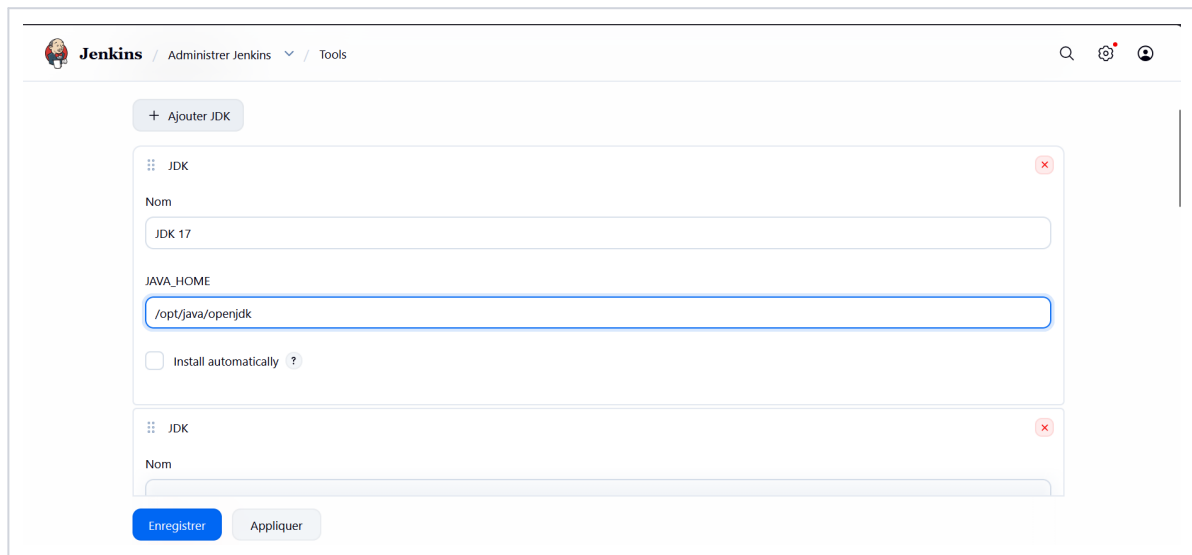
Environnement : toute l'infrastructure tourne en conteneurs Docker (Jenkins, SonarQube, PostgreSQL) via un docker compose up, sans aucune installation sur les postes. Deux branches Git sont utilisées : *avant-corrections* (code d'origine, fautif) et *main* (code corrigé).

2. Mise en place de l'environnement Jenkins

Avant de pouvoir exécuter le pipeline, Jenkins doit disposer des outils nécessaires à la compilation Java et à l'analyse de qualité : un JDK, Maven, ainsi que le plugin et le serveur SonarQube.

2.1 Configuration du JDK et de Maven

Dans *Administrer Jenkins* > *Tools*, un JDK 17 et une installation Maven sont déclarés avec leurs chemins respectifs (/opt/java/openjdk et /opt/maven), afin que le pipeline puisse invoquer mvn à chaque étape de build.

The screenshot shows the Jenkins web interface. At the top, the breadcrumb navigation reads 'Jenkins / Administrer Jenkins / Tools'. Below this is a button '+ Ajouter JDK'. The main area contains a list of configured tools. The first entry is for 'JDK'. It has a 'Nom' field with the value 'JDK 17' and a 'JAVA_HOME' field with the value '/opt/java/openjdk'. There is an unchecked checkbox for 'Install automatically' with a help icon. A second, identical entry is partially visible below. At the bottom of the configuration area are two buttons: 'Enregistrer' (highlighted in blue) and 'Appliquer'.

Déclaration du JDK 17 (JAVA_HOME) dans Administrer Jenkins > Tools.

Maven

Nom

Maven

MAVEN_HOME

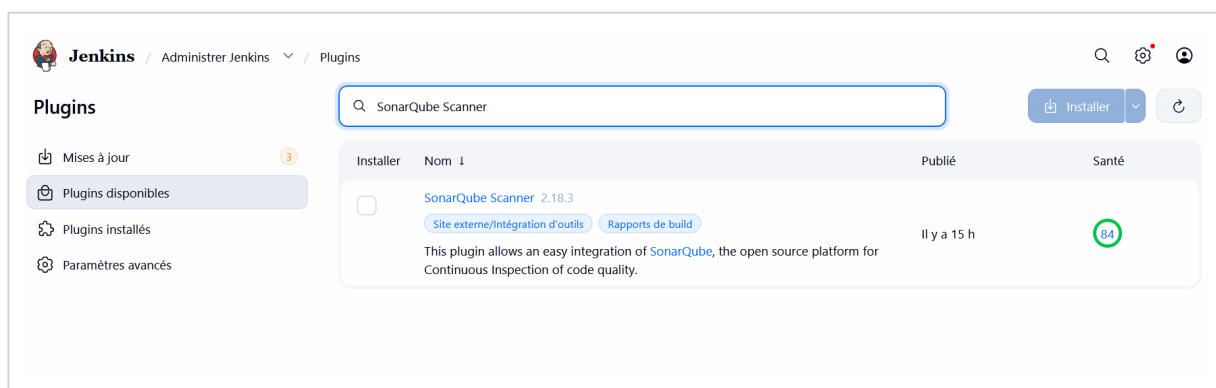
/opt/maven

☐ Install automatically ?

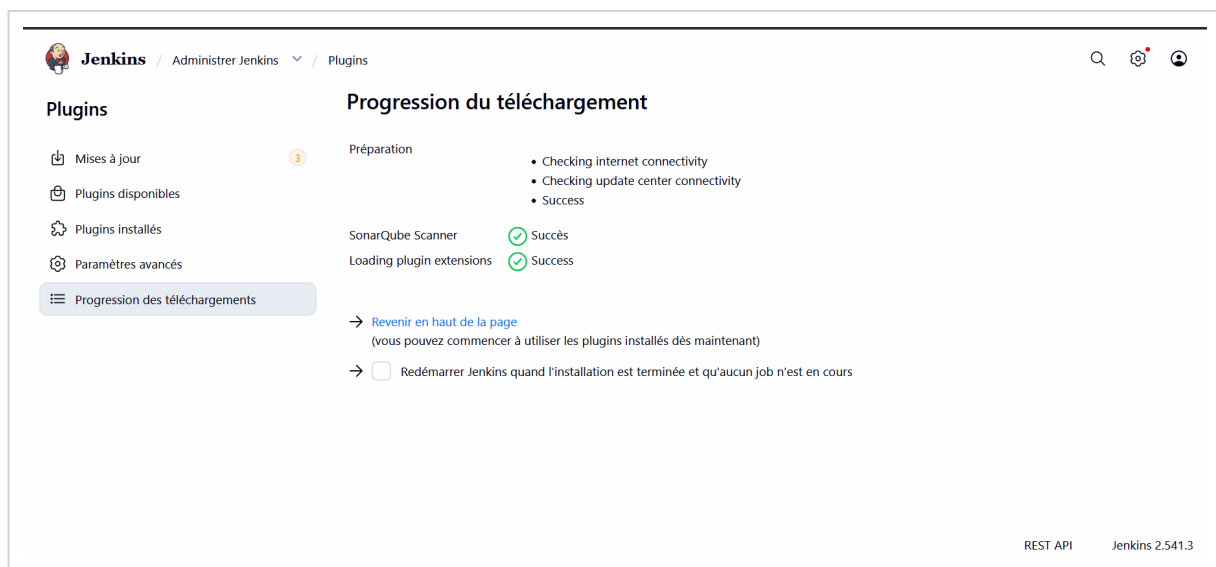
Déclaration de l'installation Maven (MAVEN_HOME) dans Administrer Jenkins > Tools.

2.2 Installation du plugin SonarQube Scanner

Le plugin **SonarQube Scanner** est installé depuis le gestionnaire de plugins Jenkins : il permet d'invoquer l'analyse `mvn sonar:sonar` et d'attendre le verdict du Quality Gate directement depuis le pipeline.



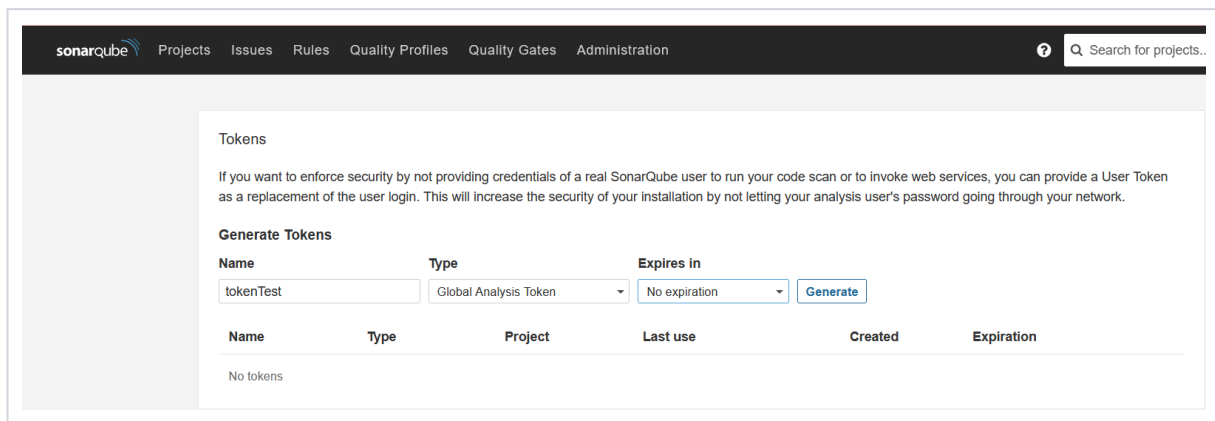
Recherche et sélection du plugin SonarQube Scanner dans le catalogue de plugins Jenkins.



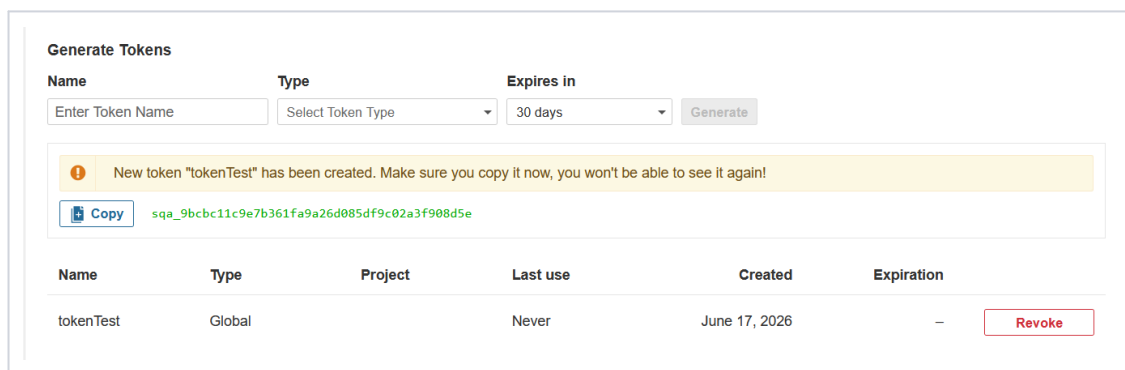
Installation du plugin SonarQube Scanner confirmée (statut « Succès »).

2.3 Génération du token d'authentification SonarQube

Pour que Jenkins puisse envoyer les résultats d'analyse sans exposer un mot de passe utilisateur, un **token d'analyse global** est généré depuis l'interface SonarQube (*Mon compte > Security > Tokens*), puis copié une seule fois (il n'est plus jamais affiché ensuite).



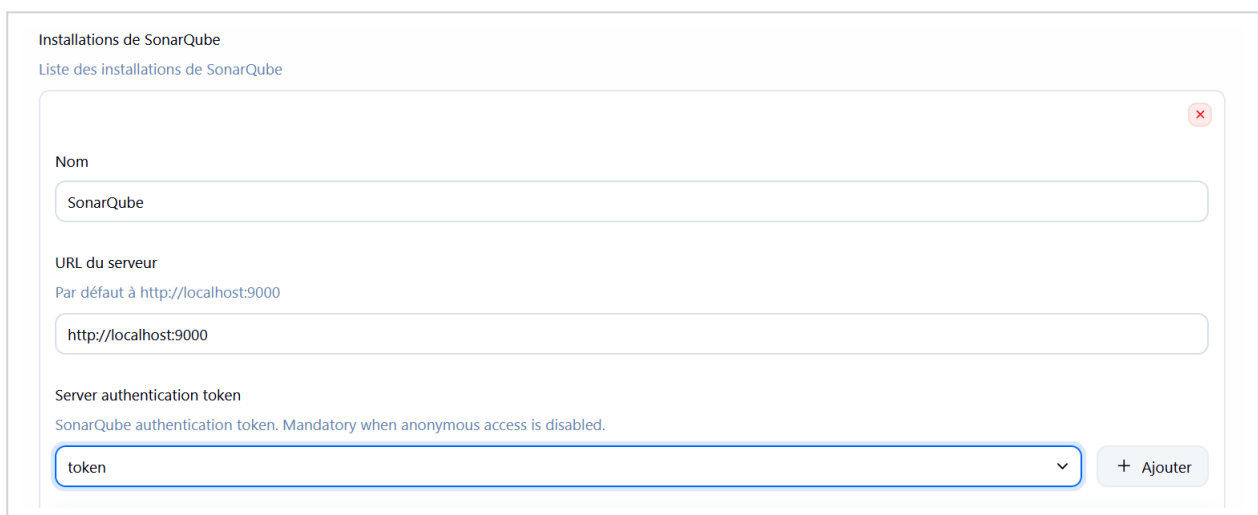
Page « Tokens » de SonarQube, avant génération : aucun token existant.



Token « tokenTest » généré avec succès — à copier immédiatement.

2.4 Interconnexion Jenkins ↔ SonarQube

Le serveur SonarQube est ensuite déclaré dans *Administrer Jenkins* > *System* > *SonarQube servers*, avec son URL (<http://localhost:9000>) et le token généré précédemment, stocké comme credential Jenkins sécurisé.



Configuration de l'installation SonarQube dans Jenkins : URL du serveur et token d'authentification.

3. Création du pipeline Jenkins

Un nouvel item de type **Pipeline**, nommé `pipe line-ci`, est créé depuis l'écran d'accueil Jenkins.

Jenkins / Tous / Nouveau Item

Nouveau Item

Saisissez un nom

pipeline-ci

Select an item type

- Pipeline**
Organise des activités de longue durée qui peuvent s'étendre sur plusieurs agents de construction. Adapté pour la création des pipelines (anciennement connues comme workflows) et/ou pour organiser des activités complexes qui ne s'adaptent pas facilement à des tâches de type libre.
- Construire un projet free-style
Job legacy polyvalent qui récupère l'état depuis un outil de gestion de version au plus, exécute les étapes de build en série, suivi d'étapes post-construction telles que l'archivage d'artefacts et l'envoi de notifications par e-mail.
- Construire un projet multi-configuration
Adapté aux projets qui nécessitent un grand nombre de configurations différentes, comme des environnements de test multiples, des binaires spécifiques à une plateforme, etc.

OK

Création du nouvel item « pipeline-ci » de type Pipeline.

Le pipeline est configuré en mode « *Pipeline script from SCM* » : Jenkins va chercher le Jenkinsfile directement dans le dépôt GitHub du projet (github.com/Kfilalimdarhri/Projet-CI), sur la branche main, ce qui garantit que la définition du pipeline reste versionnée avec le code.

Jenkins pipeline-ci / Configurer

Configurer

- General
- Triggers
- Pipeline**
- Advanced

Pipeline script from SCM

SCM

Git

Repositories

Repository URL

https://github.com/Kfilalimdarhri/Projet-CI

Credentials

- anon -

+ Ajouter

Avance

+ Add Repository

Branches to build

Branch Specifier (blank for 'any')

*/main

+ Add Branch

Navigateur de la base de code

(Auto)

Additional Behaviours

+ Ajouter

Script Path

Jenkinsfile

☒ Lightweight checkout

Pipeline Syntax

Advanced

Save Appliquer

Configuration du pipeline : SCM Git, URL du dépôt, branche main et chemin du Jenkinsfile.

4. Le pipeline Jenkins

Le pipeline est décrit dans un Jenkinsfile versionné avec le code. Il s'exécute automatiquement à chaque push et enchaîne six étapes, dont deux peuvent stopper le build (compilation et Quality Gate) :

Étape	Commande	Rôle
1. Checkout	checkout scm	Récupère le code depuis GitHub.
2. Build	mvn clean compile	Compile le code Java. Échec ⇒ build rouge.

Étape	Commande	Rôle
3. Test & Coverage	mvn test	Lance les tests unitaires ; JaCoCo mesure la couverture.
4. Analyse SonarQube	mvn sonar:sonar	Envoie le code + la couverture à SonarQube pour audit.
5. Quality Gate	waitForQualityGate	Attend le verdict de Sonar. Statut « Failed » ⇒ build rouge (arrêt).
6. Package & Docker	mvn package + docker build	Produit le JAR et l'image Docker (seulement si tout est vert).

5. État des lieux initial — PIPELINE ROUGE

PIPELINE ROUGE — Quality Gate : Failed

Premier passage du pipeline sur la branche *avant-corrections* : le build échoue au stade du Quality Gate. SonarQube remonte une faille de sécurité (le mot de passe en dur) ainsi que plusieurs code smells. Le Quality Gate est au statut « Failed », ce qui bloque toute la suite de la chaîne, comme le montre l'historique des builds (#1 à #3 en échec).

6. Journal des corrections

Chaque problème remonté par SonarQube, son impact, et la modification apportée au code Java :

Problème détecté	Type	Pourquoi c'est un risque	Correction appliquée
Mot de passe de base de données en dur (DB_PASSWORD = "super_secret...")	Faible critique	Quiconque accède au dépôt (fuite GitHub, stagiaire) obtient le mot de passe de production. Impossible à changer sans recompiler.	Suppression du secret du code : il est désormais lu depuis une variable d'environnement (System.getenv("DB_PASSWORD")).
Injection SQL par concaténation ("... WHERE username = '" + username + "'")	Faible	Un utilisateur malveillant peut injecter du SQL et lire/détruire la base.	Passage à une requête préparée (PreparedStatement avec ?), les entrées ne sont plus interprétées comme du code.
System.out.println(...) pour journaliser	Code smell	Pas de niveaux de log, pas d'horodatage, pas de traçabilité en production.	Remplacé par un java.util.logging.Logger (niveaux info / warning / severe).
Code mort : variables inutilisées (unusedCounter, isLoggedIn), imports inutiles, division par zéro factice, méthode complexMethod	Code smell	Augmente la complexité, masque les vrais bugs, rend la maintenance plus coûteuse.	Suppression complète du code mort et de la méthode trop complexe.
Blocs catch vides + gestion manuelle des ressources (finally)	Code smell	Les erreurs sont avalées en silence ; risque de fuite de connexions.	Gestion via try-with-resources (fermeture automatique) et journalisation des exceptions.

Exemple clé : la faille critique (mot de passe en dur)

Avant — le secret est écrit en clair dans le code source :

```
// UserService.java (avant)
private String DB_PASSWORD = "super_secret_password_123!";
...
conn = DriverManager.getConnection(url, "root", DB_PASSWORD);
```

Après — le secret sort du code, lu depuis l'environnement :

```
// UserService.java (après)
String dbPassword = System.getenv("DB_PASSWORD");
...
try (Connection conn = DriverManager.getConnection(DB_URL, DB_USER, dbPassword)) { ... }
```

Exemple : injection SQL

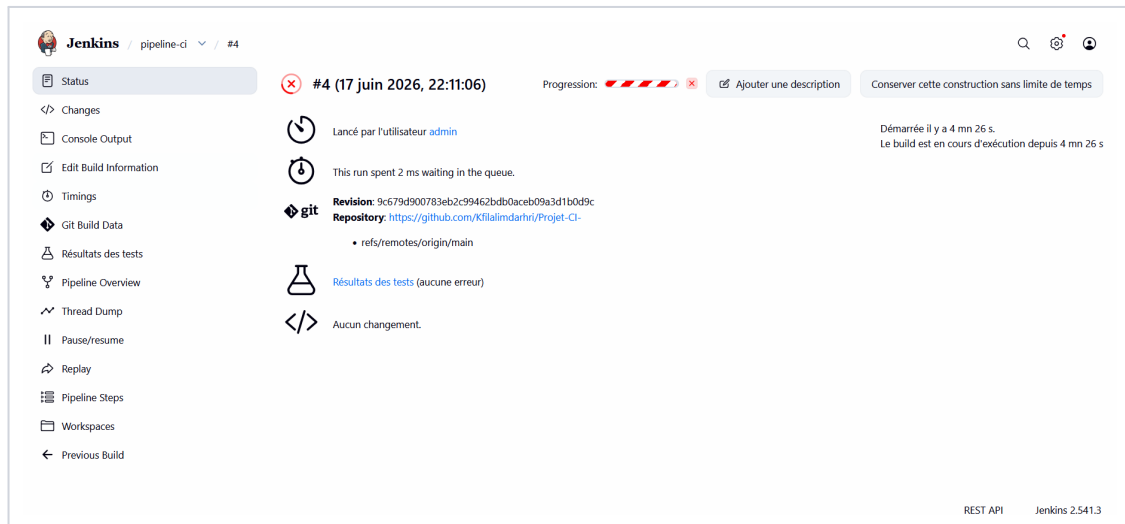
```
// avant – concaténation, injection possible
String query = "SELECT * FROM users WHERE username = '" + username + "'";
rs = stmt.executeQuery(query);

// après – requête paramétrée
String query = "SELECT * FROM users WHERE username = ?";
PreparedStatement stmt = conn.prepareStatement(query);
stmt.setString(1, username);
```

7. Résultat final — PIPELINE VERT

PIPELINE VERT — Quality Gate : Passed

Après corrections (branche *main*), le pipeline se déroule entièrement : compilation, tests, analyse SonarQube, Quality Gate, puis création du JAR et de l'image Docker. La faille de sécurité a disparu, les code smells sont résolus, et le Quality Gate passe au vert (« Passed »). Le build Jenkins est en succès sur tous ses stages.



Build #4 (branche *main*) en cours d'exécution : aucune erreur de test signalée à ce stade, le pipeline progresse normalement vers le Quality Gate.

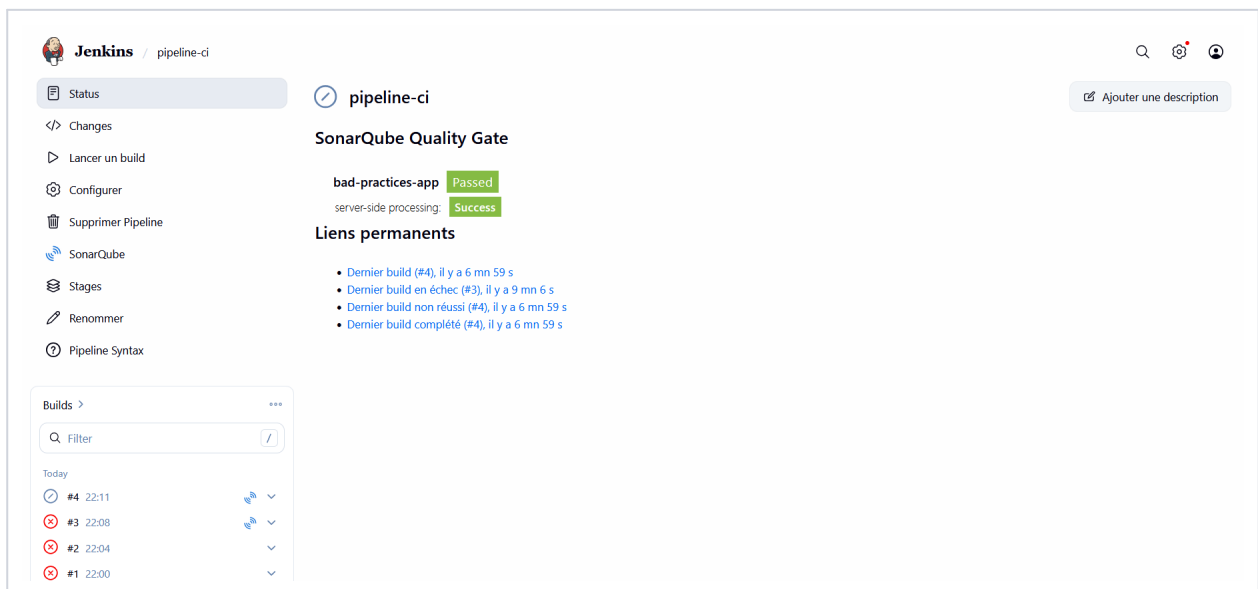


Tableau de bord du pipeline-ci après corrections : SonarQube Quality Gate au statut « Passed » / « Success » pour les deux projets analysés (*bad-practices-app* et *server-side processing*).

L'historique des builds confirme la progression : les builds #1 à #3 (branche fautive) sont en échec, tandis que le build #4, exécuté après corrections, aboutit avec un Quality Gate validé.

Le pipeline reste stable dans la durée : plusieurs exécutions ultérieures (jusqu'au build #8) continuent de réussir, confirmant que les corrections tiennent dans le temps et que le Quality Gate ne régresse pas à chaque nouveau push.

S	M	Nom du projet ↓	Dernier succès	Dernier échec	Dernière durée
✓	☀	pipeline-ci	1 mn 41 s #8	1 j 10 h #3	1 mn 29 s

État du lanceur de compilations
(0 of 2 executors busy)

Vue d'ensemble du job pipeline-ci : le dernier succès (build #8) est très récent (1 mn 41 s), alors que le dernier échec (build #3) remonte à 1 jour 10 h — preuve de la stabilité du pipeline après corrections.

8. Bonus — Conteneurisation Docker

Le pipeline produit une image Docker `epsi/bad-practices-app:latest` prête à déployer, à partir du Dockerfile du projet. Le conteneur est exécutable localement (`docker run`) et vérifiable via `docker ps`, garantissant un déploiement reproductible d'un environnement à l'autre.

9. Bilan CI/CD

Au-delà de la correction de ce code précis, le pipeline mis en place est un garde-fou permanent. À chaque push, il recompile, exécute les tests, audite la qualité et la sécurité, et refuse automatiquement (build rouge) tout code qui réintroduirait une faille ou une mauvaise pratique. L'équipe de développement conserve sa vitesse, mais ne peut plus mettre la production en danger sans que la chaîne le détecte et le bloque. La conteneurisation garantit en prime un déploiement reproductible, identique d'un poste à l'autre.